

软件工程建模

面向对象编程 · 2026 · 南京大学

| 刘钦

Outline

- 数学建模 — 问题求解五步法
- 计算机建模 — 软件框架 · 硬件框架 · 命令式 vs 函数式
- 软件工程建模 — 真实世界的复杂性 · 三级跃迁 · SE思维框架
- 软件开发生命周期模型 — SDLC · 开发活动
- 软件工程建模分析案例 — 需求 · 用例 · 图书管理系统

所有软件工程师都在思考的问题

- What to develop?
- Why to develop?
- How to develop?

思维的演化：数学 → 计算机 → 软件工程

数学建模

问题求解五步法

问题求解

问题空间

解决方案

解空间

|
求1到100这100个
数的和等于多少?

|
?

|
5050

数学的思维

问题空间

|

求 $1+2+3+\dots$

$+99+100$

等于多少?

数学模型

|

?

解空间

|

5050

数学建模五步法

步骤	内容	案例
Step 0 明确武器	数学框架：整数、加法、乘法	我们有哪些数学工具？
Step 1 审题	提炼数学问题	$results = 1+2+3+...+99+100$
Step 2 建模	建立数学模型	50对和为101的数列 (1+100, 2+99, 3+98...)
Step 3 制订方案	求解	$result = 50 * 101$
Step 4 检查	Verification	whether do it right?
Step 5 实施	计算	$result = 5050$

计算机建模

软件框架 · 硬件框架 · 指令执行

计算机与数学的关系

- 数学为计算机提供了理论基础
 - 现实世界的问题先转换为一个数学问题
 - 然后再用计算机解决这个数学问题
- 计算机有特有的软件和硬件实现
 - 软件框架（编程范式：命令式、函数式）
 - 硬件框架（硬件结构：冯诺依曼结构、哈佛结构）

软件框架

- 编程范式
 - 命令式
 - 函数式
 - 逻辑式
- 层次性
 - 机器指令
 - 汇编指令
 - 高级语言

计算机的思维

问题空间

|

求 $1+2+3+\dots$
 $+99+100$
等于多少?

数学模型

|

?

计算机模型

|

?

解空间

|

5050

计算机建模五步法（命令式）

步骤	内容
Step 0 明确武器	计算机框架：软件（命令式编程语言、汇编器）+ 硬件（寄存器、ALU）
Step 1 审题	results = 1+2+3+...+99+100
Step 2 建模	循环100次，一个寄存器保存新加的数，一个寄存器保持累加和，计算加法，将累加和送入内存
Step 3 制订方案	x86汇编程序
Step 4 检验	Verification
Step 5 实施	编译+执行，result = 5050

制订解决方案 — x86 汇编

```
.model small
.stack
.data
    sum dw ?
.code
.startup
    xor ax, ax        ; 被加数AX清0
    mov cx, 100
again:
    add ax, cx        ; 从100,99,...,2,1倒序累加
    loop again
    mov sum, ax       ; 将累加和送入指定单元
.exit 0
end
```

为什么用数学能解决的还要用计算机做？

- 可以节省人力
- 计算的快速
- 存储的海量

不同的软件框架，解决问题是不一样的

计算机建模五步法（函数式）

步骤	内容
Step 2 建模	$\text{Sum}(n) = n + \text{Sum}(n-1)$, $\text{Sum}(1) = 1$
Step 3 制订方案	Scheme 代码

```
(define (sum n)
  (if (= n 1)
      1
      (+ n (sum (- n 1)))))
```

(sum 6) 展开过程

```
(sum 6)
(+ 6 (sum 5))
(+ 6 (+ 5 (sum 4)))
(+ 6 (+ 5 (+ 4 (sum 3))))
(+ 6 (+ 5 (+ 4 (+ 3 (sum 2)))))
(+ 6 (+ 5 (+ 4 (+ 3 (+ 2 (sum 1))))))
(+ 6 (+ 5 (+ 4 (+ 3 (+ 2 1)))))
(+ 6 (+ 5 (+ 4 (+ 3 3))))
(+ 6 (+ 5 (+ 4 6)))
(+ 6 (+ 5 10))
(+ 6 15)
21
```

SSE 技术 — 不同硬件框架，不同解法

- 1999年，Intel在其Pentium III微处理器中集成了SSE（Streaming SIMD Extensions）技术
- SSE兼容MMX指令，可以通过SIMD和单时钟周期并行处理多个浮点数据
- 具有SSE指令集支持的处理器有8个128位的寄存器，每一个寄存器可以存放4个单精度(32位)浮点数

SSE 案例

问题： 计算一个很长的浮点型数组中每一个元素的平方根

传统方案

```
for each f in array {  
    把f从主存加载到浮点寄存器  
    计算平方根  
    再把结果从寄存器写入主存  
}
```

基于SSE的方案

```
for each 4 members in array {  
    把4个数加载到128位SSE寄存器  
    在一个CPU指令周期中完成  
        4个数的平方根操作  
    把4个结果取出写入主存  
}
```

软件工程建模

从编程到工程

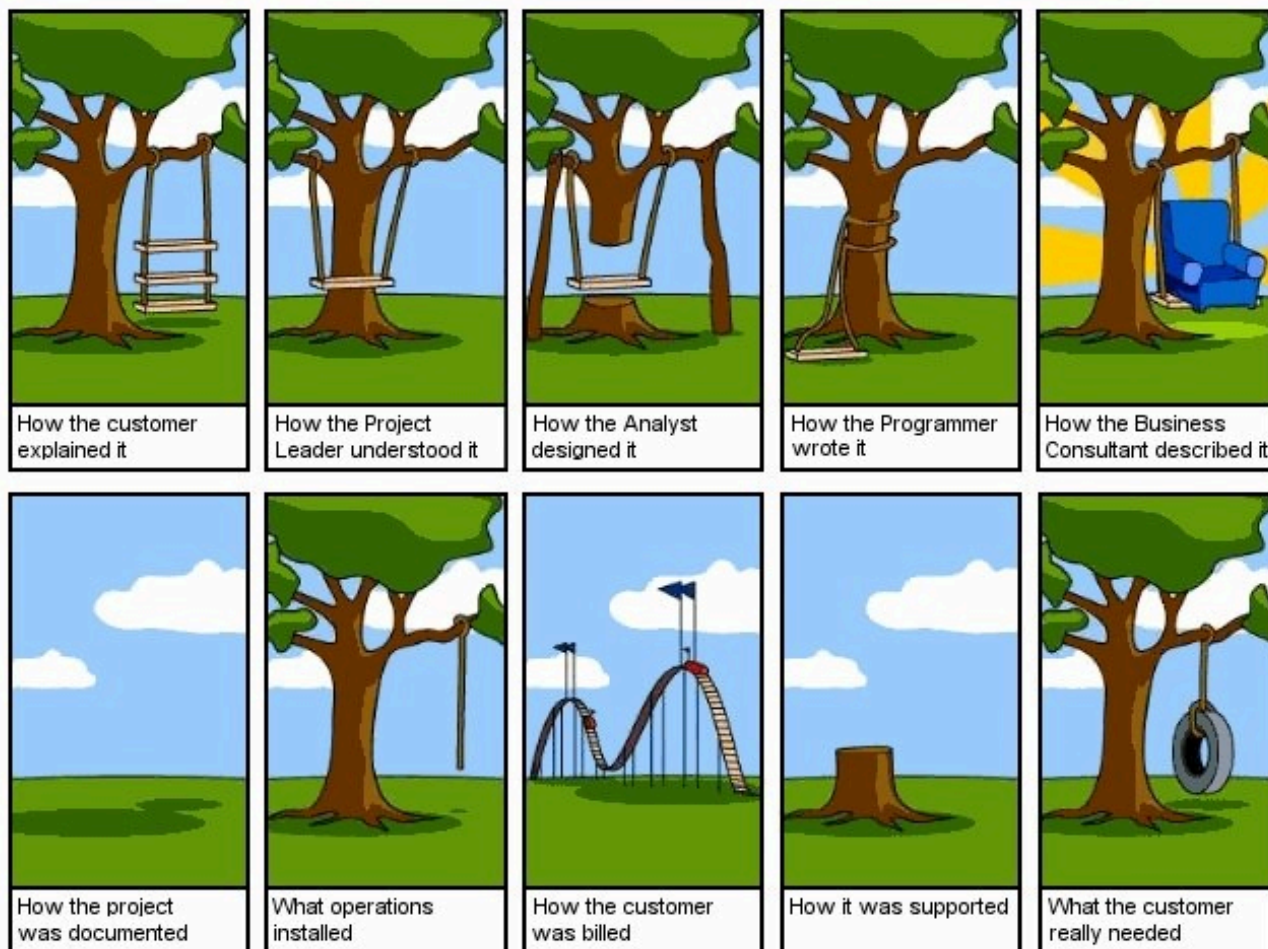
真实世界就是这么简单？

- 我们需要一个帮我们计算 $1+2+3...+100$ 的软件？
- 到底我们的需求是什么？ → 我们想要一个好用的计算器软件。

从需求到交付的一系列问题

- 为什么需要这样一个软件？
- 到底什么是一个好用的计算器？
- 怎么实现这个好的计算器？
- 怎么知道实现的这个计算器软件是我们需要的那个？怎么知道他是好用的？
- 现在的软件已经复杂到不是一个人能够开发出来的。
- 所以，如果要求在一定的时间，一定预算内，领导特定一群人来求解空间？怎么办？

What the customer wanted



沟通断裂是软件项目失败的首要原因。

这时候你需要软件工程！

软件危机（1968）

1968年 NATO 软件工程会议正式提出**"软件危机"**:

表现	说明
超预算	项目实际花费远超预期（OS/360：预算\$25M → 实际\$100M+）
超工期	交付日期一再延期
质量差	交付的软件充满缺陷，不满足用户需求
难维护	代码无法理解、无法修改、无法扩展
项目取消	大量项目在完成前被放弃

编程 ≠ 软件工程。写出能运行的代码只是工程的一小部分。

软件工程的定义

IEEE 610.12-1990：软件工程是将系统化的、规范化的、可量化的方法应用于软件的开发、运行和维护的过程。

关键词	含义
系统化	有组织的、有计划的，不是随意拼凑
规范化	遵循标准和规范（IEEE/ISO/CMMI）
可量化	可以度量进度、质量、成本

软件的本质复杂性 (Brooks, 1987)

"No Silver Bullet — Essence and Accidents of Software Engineering"

本质复杂性	说明
复杂性 (Complexity)	软件实体比任何人类构造物都更复杂，没有两部分是相同的
一致性 (Conformity)	软件必须适配外部环境（硬件、法规、已有系统）的任意要求
可变性 (Changeability)	软件不断面临变更压力——成功的软件会被要求做更多
不可见性 (Invisibility)	软件没有物理形态，无法用空间图形自然表达

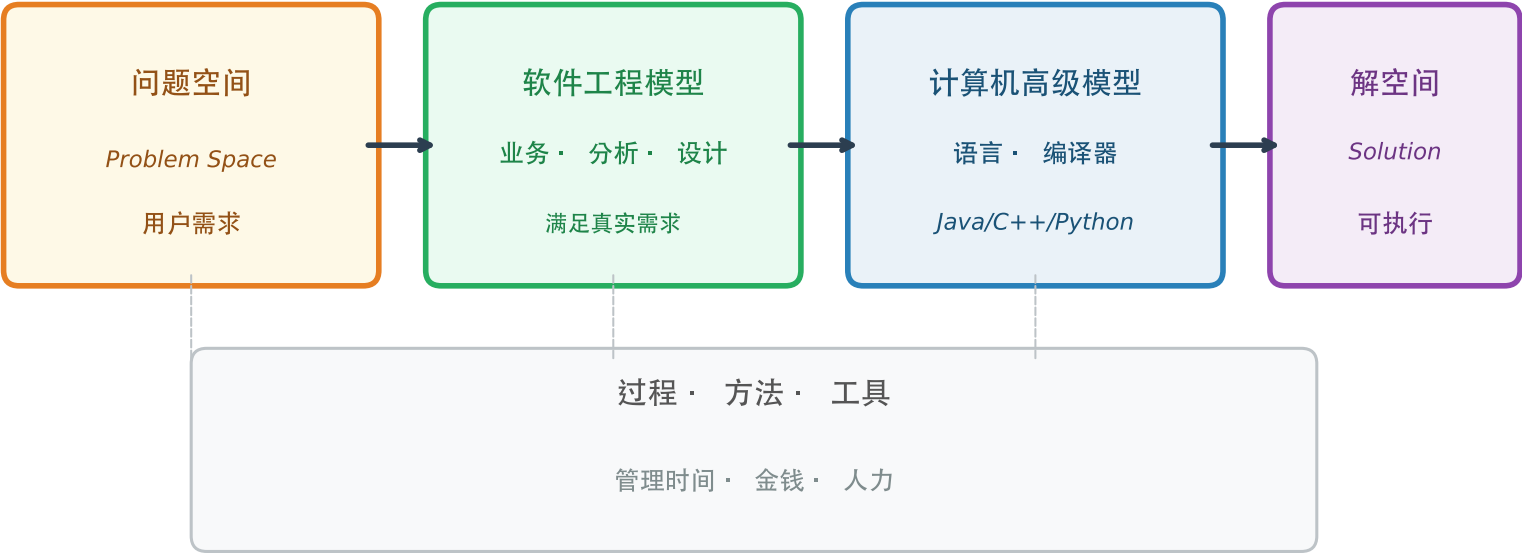
这四种复杂性是本质的，无法通过任何工具或方法消除——只能管理。

软件工程的思维框架

Step 0 ~ Step 5

软件工程的思维

软件工程的思维 — 从问题到解



软件工程的本质：在**问题空间**和**解空间**之间搭建**受控的桥梁**——用过程、方法、工具管理时间、金钱、人力，让真实的用户需求可以被系统化地转化为可执行的软件。

第零步 – 明确自己的武器

- 软件工程框架
 - 技术
 - 业务模型
 - 分析模型
 - 设计模型
 - 过程
 - 软件开发过程
- 计算机模型的进步
 - 高级语言编译器
 - 高级的模型

五步流程总览



每一步都有明确的产出物（artifact）——上一步的产出是下一步的输入，形成清晰的工程链路。

第一步 – 需求分析（审题）— 做什么？

分析模型 — 业务描述（图书管理系统）

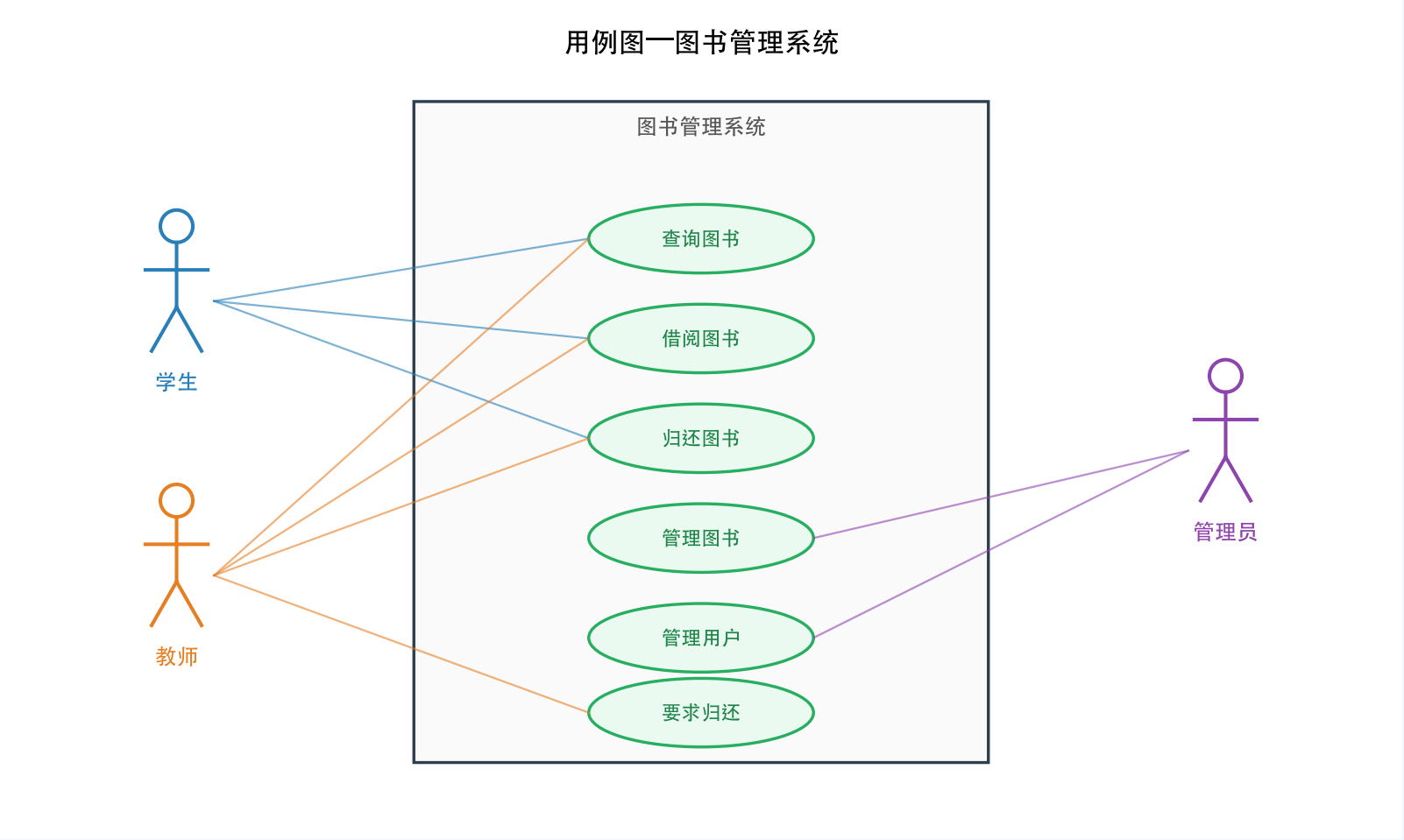
- 系统管理员

- 身份：所有系统管理员采用相同的身份和权限。
- 操作：
 - 管理系统管理员：查询、添加、修改和删除
 - 管理借阅人：查询、添加、修改和删除
 - 管理图书：查询、添加、修改和删除

- 借阅人

- 身份：本科生、研究生和教师三种身份，各种身份具有不同的权限。
- 操作：
 - 查询图书
 - 借阅图书（本科生5本/研究生10本/教师20本）
 - 请求图书 · 查看已借图书 · 续借图书
 - 归还图书 · 查看消息

分析模型 — 用例图



用例图展示系统的**参与者（Actor）和用例（Use Case）**之间的关系——椭圆是用例，小人是参与者，连线表示"谁能做什么"。

分析模型 — 分析类图

图书类别

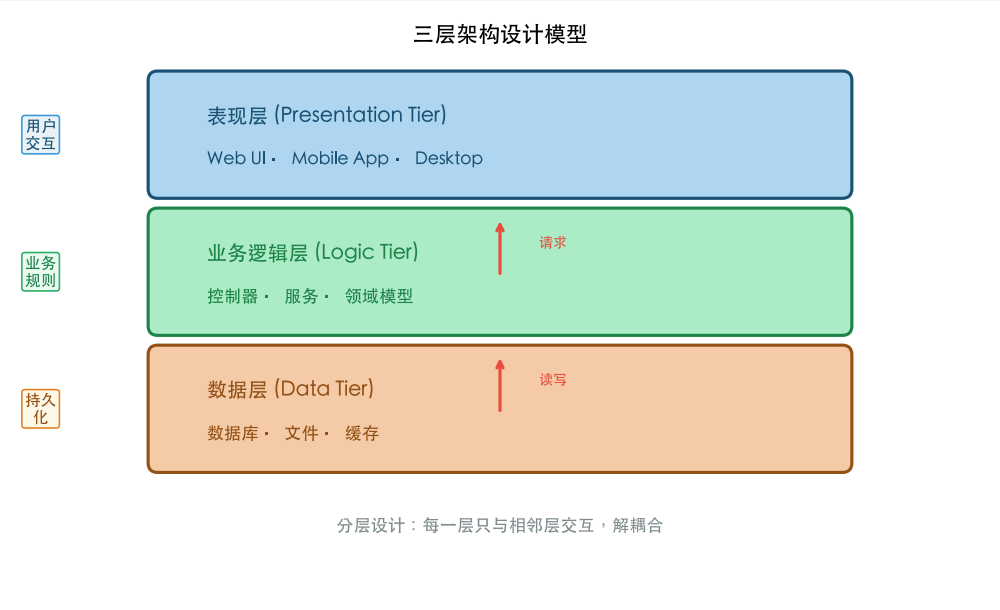
- -图书信息列表
- -图书计数
- +获取图书数量()
- +获取单本图书()
- +根据ISBN搜索图书()
- +根据基本信息搜索图书()
- +添加图书()
- +删除图书()

借阅人

- -编号
- -姓名
- -密码
- -借阅列表
- -借阅数量
- -借阅数量限制
- +获取个人属性信息()
- +修改个人信息()
- +列举所借图书()
- +借阅图书()
- +归还图书()

第二步 – 设计（建立计算机模型） — 怎么做

设计模型 — 三层架构



核心思想

分层解耦：每一层只与相邻层交互，上层**依赖**下层，下层**不知道**上层存在。

三层各自的职责

层	职责
表现层	用户交互 · UI 渲染
业务逻辑层	规则 · 流程 · 领域模型
数据层	持久化 · 缓存

修改 UI 不影响业务规则，更换数据库不影响上层逻辑。

设计模型 — 其他常用设计图

视图	回答的问题	何时使用
类图 (Class Diagram)	系统由哪些类组成？类之间是什么关系？	静态结构设计
序列图 (Sequence Diagram)	对象之间如何交互？消息传递的时序？	动态行为分析
组件图 (Component Diagram)	系统由哪些可部署的组件组成？	部署架构设计
活动图 (Activity Diagram)	业务流程是怎样的？分支和并发？	业务流程建模
状态图 (State Diagram)	对象有哪些状态？状态如何迁移？	有状态系统建模

UML 四大类图：用例图（需求阶段）· 类图 + 对象图（静态结构）· 序列图 + 通信图（动态交互）· 组件图 + 部署图（物理架构）。

第三步 – 构造（制订解决方案）

- Do it
- 构造模型 — 高级语言语法 (Java / C++ / Haskell)

```
class HelloWorldApp {  
    public static void main(String[] args) {  
        System.out.println("Hello World!");  
    }  
}
```

第四步 – 软件测试（检验）

	Verification	Validation
问题	Are we building the product right ?	Are we building the right product?
关注	过程正确性：代码是否符合设计规格	结果正确性：软件是否满足用户真实需求
方法	代码审查、单元测试、静态分析	验收测试、用户测试、原型演示
发现	缺陷（Bug）	需求偏差

Verification 保证"做对了", Validation 保证"做了对的东西"——两者缺一不可。

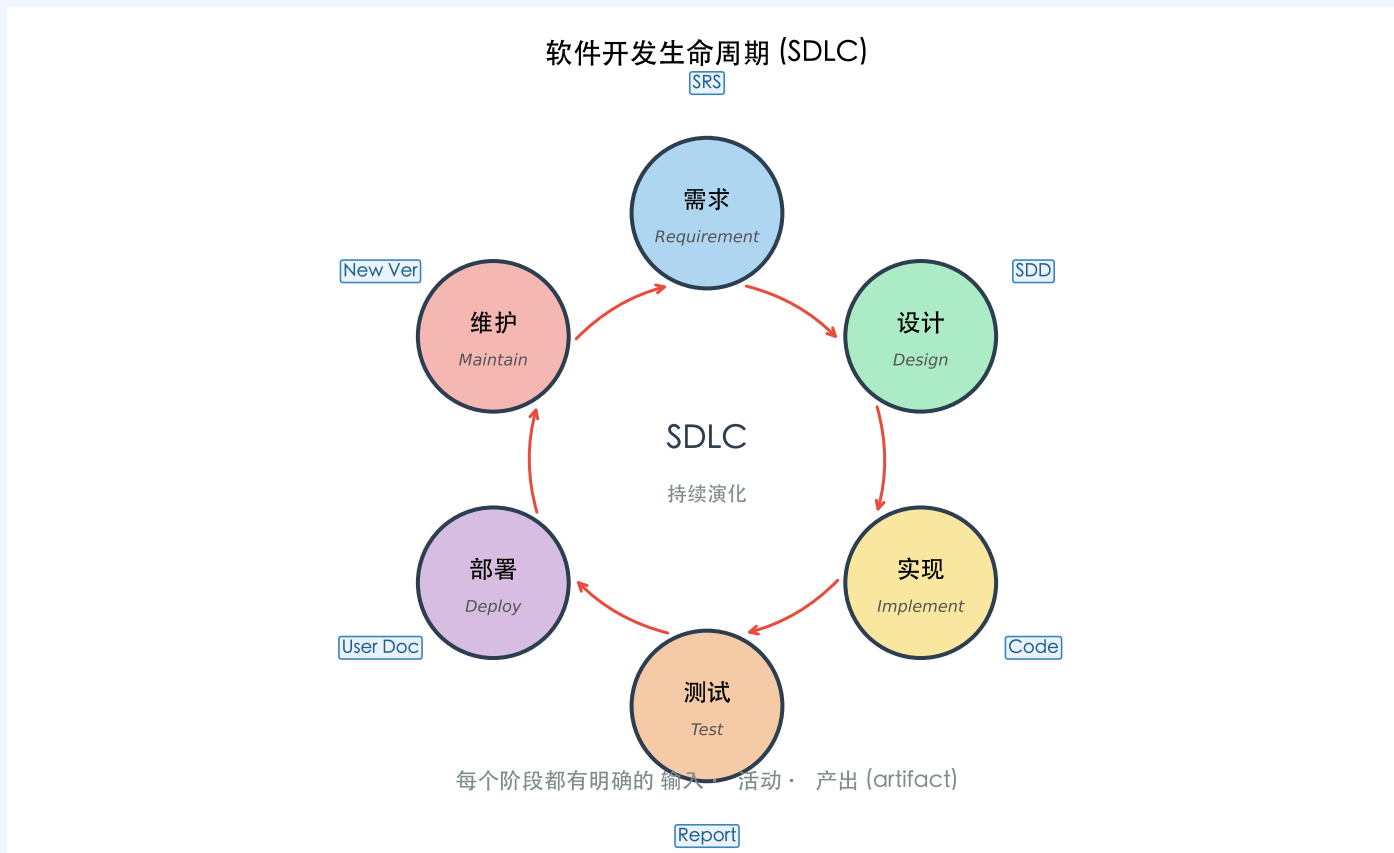
第五步 – 移交和演化（实施）

- 在真实环境中运行
- 演化新的版本

软件开发生命周期模型

SDLC

SDLC (Software Development Life-Cycle)



SDLC 是一个循环：部署后进入维护，维护中产生新需求 → 回到起点——软件永远在演化。

Software Development Activity

Activity	Target	Artifact
Software Requirement	What	SRS
Software Design	How	SDD
Software Construction	Build	Code and executable file
Software Testing	Right thing? Right?	Test Report
Software Deliver	Install	User/System Document
Software Maintenance	Revolution	New version software

软件过程模型对比

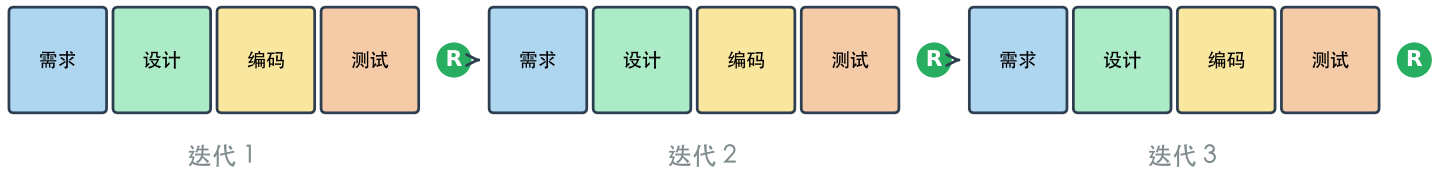
软件过程模型对比

瀑布
Waterfall

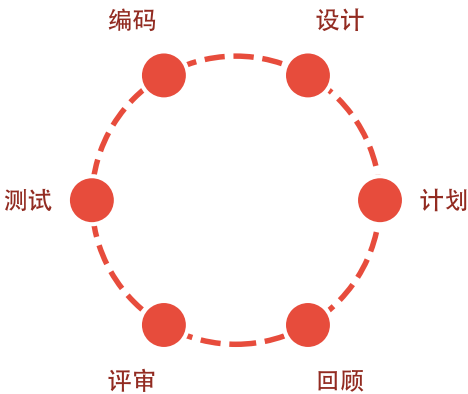


R = Release
(可运行版本)

迭代
Iterative



敏捷
Agile



2-4 周/迭代
拥抱变化
持续交付

软件工程建模分析案例

需求 · 用例 · 图书管理系统

如何给软件需求建模？ — 什么是需求？

IEEE对需求的定义为[IEEE610.12-1990]:

- (1)**用户**为了解决问题或达到某些目标所需要的条件或能力；
- (2)**系统**或系统部件为了满足合同、标准、规范或其它正式文档所规定的要求而需要具备的条件或能力；
- (3)对(1)或(2)中的一个条件或一种能力的一种**文档化**表述。

需求是一种**期望**

如何表达这种期望？

- 一段录音？一段文字？
- 如何组织这些内容？

用例

- 用例最初由[Jacobson1992]在 Objectory 方法中提出的，它将用例定义为"在系统(或者子系统或者类)和外部对象的交互当中所执行的行为序列的描述，包括各种不同的序列和错误的序列，它们能够联合提供一种有价值的服务"[Rumbaugh2004]。
- [Cockburn2001]认为用例描述了在不同条件下系统对某一用户的请求的响应。根据用户的请求和请求时的系统条件，系统将执行不同的行为序列，每一个行为序列被称为一个场景。一个用例是多个场景的集合。

寻找参与者

- 谁对系统有着明确的目标和要求并且主动发出动作？
- 系统是为谁服务的？

用例的特征

- 用例是相对独立的
 - 取钱？填写取款单？
- 用例的执行结果对参与者来说是可观测的和有意义的
 - 登陆系统？后台进程监控？
- 这件事必须有一个参与者发起
 - ATM吐钞票？
- 用例必须是以动宾短语形式出现的
 - 统计？报表？
- 一个用例就是一个需求单元、分析单元、设计单元、开发单元、测试单元，甚至部署单元

目标和步骤

- 参与者：寄信人
- 用例：
 - 买信封
 - 买邮票
 - 付钱
 - 投递

用例文本描述

字段	说明
ID	用例的标识
名称	对用例内容的精确描述, "动词 + 名词"
参与者	主参与者、辅助参与者和每个参与者的目标
触发条件	标识启动用例的事件
前置条件	用例能够正常启动和工作的系统状态条件
后置条件	用例执行完成后的系统状态条件
正常流程	在常见和符合预期的条件下, 系统与外界的行为交互序列
扩展流程	用例中可能发生的非常见的其他合理场景
特殊需求	和用例相关的其他特殊需求, 尤其是非功能性需求

画一个图书管理系统的用例图！

性能分析 — String 类的使用

String 类的使用

不可变类 (Immutable)

- 不可改变的字符串具有一个很大的优点：编译器可以把字符串设置为共享。
- JAVA为了提高效率，所以对于String类型进行了特别的处理——为String类型提供了**串池**。定义一个string类型的变量有两种方式：
 - `String name = "tom";` — 使用了串池
 - `String name = new String("tom");` — 普通的声明对象的方式
 - 使用第一种方式，如果你使用了第一种方式，那么当你再声明一个内容也是 "tom" 的String时，它将使用串池里原来的那个内存，而不会重新分配内存，将会指向同一块内存
- String类型是**不可改变的**，当你想改变一个String对象的时候，比如 `name = "madding"`，那么虚拟机不会改变原来的对象，而是生成一个新的String对象，然后让name去指向它。如果原来的那个 "tom" 没有任何对象去引用它，虚拟机的垃圾回收机制将接收它。

性能对比：String "+" vs StringBuffer vs StringBuilder

Case 1 — String "+"

```
String tempstr = "abcdefghijklmnopqrstuvwxyz";
int times = 5000;
long lstart1 = System.currentTimeMillis();
String str = "";
for (int i = 0; i < times; i++) {
    str += tempstr;
}
long lend1 = System.currentTimeMillis();
System.out.println(lend1 - lstart1);
```

Case 2 — StringBuffer

```
StringBuffer sb = new StringBuffer();
for (int i = 0; i < times; i++) {
    sb.append(tempstr);
}
```

Case 3 — StringBuffer (预分配)

```
StringBuffer sb = new StringBuffer(
    tempstr.length() * times);
for (int i = 0; i < times; i++) {
    sb.append(tempstr);
}
```

性能测试结果

方式	100,000次 (Capacity:16)	100,000次 (Capacity:100000)
concatStrBuff (StringBuffer)	length: 100000 time: 16	time: 15
concatStrBuild (StringBuilder)	length: 100000 time: 16	time: 15
concatStrAdd (String +)	length: 100000 time: 10437	time: 10594

StringBuilder 比 StringBuffer 更快，因为它避免了 synchronization。String + 比两者慢 600+ 倍！

char[] 与 String

```
String strPassword = "Unknown";  
char[] charPassword = new char[]{'U','n','k','w','o','n'};  
  
System.out.println("String password: " + strPassword);  
System.out.println("Character password: " + charPassword);
```

- Output:
 - String password: Unknown
 - Character password: [C@110b053

char[] 打印的是引用地址，而非内容——这在安全场景中是一个优势（密码不应该以String形式存在内存中）。

String.intern()

- 存在于.class文件中的常量池，在运行期被JVM装载，并且可以扩充。String的 `intern()` 方法就是扩充常量池的一个方法。
- 当一个String实例str调用 `intern()` 方法时，Java查找常量池中是否有相同Unicode的字符串常量：
 - 如果有，则返回其的引用
 - 如果没有，则在常量池中增加一个Unicode等于str的字符串并返回它的引用

```
String s0 = "kvill";
String s1 = new String("kvill");
String s2 = new String("kvill");
System.out.println(s0 == s1);           // false
s1.intern();                             // 返回值未赋值
s2 = s2.intern();                       // 把常量池中引用赋给s2
System.out.println(s0 == s1);           // false (s1未改变)
System.out.println(s0 == s1.intern());  // true
System.out.println(s0 == s2);           // true
```

Java对象在JVM中的存储

- 一般而言，Java 对象在虚拟机的结构如下：
 - 对象头 (object header)：8 个字节
 - Java 原始类型数据：如 int, float, char 等类型的数据
 - 引用 (reference)：4 个字节
 - 填充符 (padding)

String在JVM中的存储

- 一个 Java 对象实际还会占用些额外的空间 (class 信息、ID、状态等), 在 Hotspot 中一个普通对象需要额外 8 个字节
- String (JDK 6) 的成员变量: `private final char value[]; private final int offset; private final int count; private int hash;`
- 一个空 String 所占空间为:
 - 对象头(8) + char数组(16) + 3个int(3×4=12) + 1个char数组引用(4) = **40 字节**
- String所占空间公式: `8 * ((8+12+2*n+4+12)+7) / 8` 字节 (n为字符串长度)

案例：大规模文本分析的内存问题

- 程序需要统计一个 300MB 的 csv 文件所有单词的出现次数
- 分析发现共有 20,000 左右的唯一单词，假设每个单词平均包含 15 个字母
- 根据上述公式，一个单词平均占用 75 bytes，那么 $75 \times 20,000 = 1,500,000$ ，即约 **1.5M**
- 但实际发现有**上百兆**的空间被占用！

原因： `String.substring()` 所返回的 String 仍然会保存原始 String（共享 char[]），导致大量冗余数据

解决方案： `String newString = new String(smallString.toCharArray());`

String 构造的方法选择

- `String a1 = "Hello";` — JVM 查看串池是否已有相同字符串，有则返回已有引用，无则创建新的
- `String a2 = new String("Hello");` — 直接调用构造函数创建，如果串池中不存在则调用构造函数创建全新字符串，如果已有则再拷贝一份到 Java 堆中

第一种方式更高效，应优先使用。

总结

软件建模

- 三级建模跃迁：数学 → 计算机 → 软件工程
- 数学建模：五步法（明确武器→审题→建模→解决→检查→实施）
- 计算机建模：命令式(汇编) · 函数式(Scheme) · SSE(SIMD)
- 软件工程
 - | 软件危机(1968) → SE定义(IEEE) → 本质复杂性(Brooks)
 - | 思维框架：技术(业务/分析/设计) + 过程(开发过程)
 - | Step 0~5：明确武器→需求→设计→构造→测试(V&V)→移交
- SDLC：瀑布 · 迭代 · 敏捷
- 建模分析案例：需求 · 用例 · 用例图 · 用例文本描述
- 性能分析：String不可变 · StringBuffer/Builder · intern() · JVM存储

课后作业

#	任务	要求
1	用例分析	为图书管理系统识别参与者和用例，画用例图
2	用例文本	选择"借阅图书"用例，写出完整的用例文本描述（含正常流程和扩展流程）
3	反思	300 字：数学建模、计算机建模、软件工程建模三种思维方式的核心区别是什么？

下节课预告

第 10 讲：面向对象编程 I — 思想

- 对象是什么？类与实例的关系
- 封装 · 继承 · 多态三大特征
- 从"动词驱动"到"名词驱动"的思维转变

课前思考：描述一个"学生选课系统"，你会先想到哪些**名词**？哪些**动词**？哪个更稳定？